

Assignment 5: Interpretation and Simulation

Gabriel Castillo (gac232), Leo Qian (rq48), Wallace Wei (wzw4), Katherine Hu (kyh29)

Summary

In A5, we are tasked with creating a simulator that maintains the execution environment of the critter world, implementing an interpreter of the critter language, and implementing a controller interface for controlling the simulation and querying the state of execution. The most challenging part of this assignment was working with the hexagonal world. Even though the 2D arrays helped visualize and work with coordinates, it was still tricky to go from indices of the two arrays to coordinates or back. It was very tricky to think about the length of each array, since in situations where the number of columns is odd the even array should hold one more tile for every row than the odd array. And when the number of rows is odd the even array should hold one more tile for every column than the odd array. Thinking about these edge cases themselves was very tricky.

Another thing that was very challenging was writing an extensive number of tests required for the critter step() method. Considering the vast number of actions a critter can take, and the different scenarios and edge cases it could encounter when taking any given action, building a comprehensive testing plan for this method specifically was very difficult and time consuming.

An interesting design decision was to store the critter world in two 2D arrays, odd and even that store tiles in odd and even rows respectively. This implementation made working with the coordinates simpler to think about, and using two arrays instead of one prevents wasting memory storing non-existent tiles.

Specification Changes

There are a few ambiguous things that we implemented in the way we thought was most appropriate. First, when a parent critter buds, we set the orientation of its child to the opposite of its parent. This is the same if two critters reproduce, one of the parents is chosen randomly and the orientation of the child is set to the opposite of that parent. This is because first, babies are mostly born head first, and second it prevents the newborn child from mating with its mother immediately after birth. Also, it prevents the parent from eating its own baby (immediately). Also, when two critters are placed on top of each other, or two objects are placed on top of each other, the first one is kept and the second is not used.

Additionally, for the smell method, our function returns the smallest possible value of smell, not the value of the food that is nearest to the critter in terms of steps required to approach it. For example, if there is a food tile at direction 5 relative to the critter and another one at direction 3, if both are only 1 tile away our smell method will return 1003 not 1005, even though the food at direction 5 could be reached in two turns while the food at direction 3 would take 4 turns.

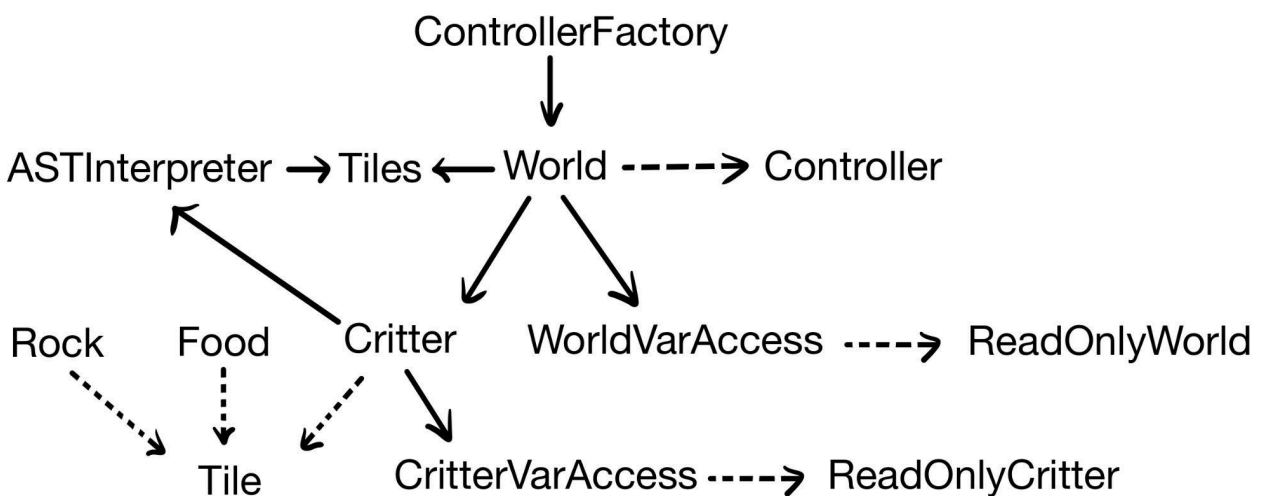
Another change was that randomly generated worlds will have a maximum of 1/3 of its tiles filled with rocks. This is to prevent the creation of worlds where it is extremely difficult to reach food or find other critters due to an abundance of rocks.

Finally, for loadCritters we check if the first lines of the file are valid (ie start with either "posture:" or "energy:" and are followed by a number) to update those values for the created critter, but if we find that one of them is invalid we assume that the rest of the file is the critter program and attempt to parse it using the parser from A4. This follows the spec as if any value is missing we fill it in with a default value and continue to parse the critter file with that default value. This also lets us parse a default critter with a file that just contains an AST, which according to the spec should be allowed.

Design and Implementation

Architecture

Here is a module dependency diagram of our program:



Note that **WorldVarAccess** and **CritterVarAccess** are nested classes within the **World** class or **Critter** class respectively. This keeps the methods separate and makes the code more organized and easier to understand and use.

Code Design

Our group decided to split functionality into 3 main classes: **Critter**, **Tiles**, **World**.

World is responsible for managing the **Critter**, **Rock**, and **Food** instance objects. Additionally, it communicates with all its **Critter** instances both ways, whether it's sending **Critter** information data for its sensory inputs, or receiving **Critter** action to change the world state. Some methods that it handles include `printWorld()` and `advanceTime()`.

Critter is responsible for storing the data of each Critter, such as its memory array, AST, age, etc. Some methods that it handles include `getComplexity()`, `mutate()` and `step()`.

Tiles is responsible for handling the hexagonal coordinate system and storing all the tiles in the world. Some methods that it handles include `ahead()`, `nearby()` and `smell()`.

We can now dive more in-depth into our code design. The World class depends on the Tiles class. The Tiles class is responsible for handling all of the tiles of the hexagonal tiled critter world. It is also responsible for implementing methods that check the validity of a tile coordinate, checking the surroundings around a tile and moving objects to different tiles. The way Tiles stores these tiles is in two 2D arrays, odd and even. These store the odd and even rows of the hexagonal grid respectively. This is because there are cases where there are one fewer tile in the odd rows than there are in the even rows. World is also responsible for implementing the ASCII print functionality for the world.

Additionally, World is responsible for the simulation, advancing time and keeping track of all critters in the world and running their action in their respective turns. To keep track of critters, World will store an ArrayList of critters which it will loop through when moving forward time steps, considering the order of execution is very important in the simulation. Given that World will be the only class to know whether a critter action was successful or not, here is how we decided to handle critter actions. Every time it is a critter's turn, its `step()` function is called. This returns an ActionType (enum) which is read by the world. The world will then execute the according action, and call an `updateEnergy()` method of the critter to update its energy depending on the success or failure of the action. This `updateEnergy()` method returns a boolean indicating whether the energy of the critter dipped to zero or lower meaning that the critter died. If this is the case the World updates the simulation as required.

We made the decision to delegate all the work of managing each critter's energy and action to World, this is because all energy changes for all Critters come from Critter actions, and since Critter actions must be processed by World to determine if that action is successful or not, we allowed World to control Critter's energy as well, for the sake of simplicity.

World also has a stack that stores all currently mating critters. This is because in order for Mating to be successful, all critters involved must've stepped. To guarantee that this is the case, we first make sure that *all* critters are stepped first. Any Critter indicating a Mating action will be added to a stack to be checked after all critters are stepped to check all mating critters against each other at once. This ensures that our simulator will simulate critter mating correctly with a relatively simple design.

When mating, a new critter is created and placed inside the world, but will not take an action, so it is stored in a separate arraylist from the other critters to make sure it doesn't take an action, called `newCritters`, which is then added to the critter list after the turn has ended.

In the Critter class, we store the coordinates of the critter, their orientation, their memory array and their implementation (the root node of their AST). As mentioned, it has a `step()` method that runs their implementation and returns a CritterAction (enum) depending on the result.

Programming

We decided on a top-down approach to creating simulation of the critter, because many modules require careful thought and planning before we actually get to coding and implementing the various actions, creating the world, etc. It is very important that we properly connect the World generation and the critters correctly, and that we are able to properly share code like by using nested classes, so that we can make sure that the implementation isn't unsafe or using too much memory/time.

Following this design strategy, our team met for a few hours before starting to code just to think about the code design of this project. We then decided on to split functionality into the following classes: Critter, Tiles, World (as mentioned), and went into each of these classes writing method stubs and variables for all the methods each class was responsible for implementing. We also wrote the JavaDoc for each of these methods and variables. This greatly facilitated this whole assignment as not only did it allow for a modular workflow where we can focus on implementing one singular method at a time, it also allowed us to delegate work really easily, and it also made writing tests significantly easier, as we could begin writing tests early on and prevent having to spend copious amounts of time trying to find the root of a bug in a near finished program.

Testing

When writing testing, our priority was to test our Critter Interpreter as soon as possible, because many of the tests for other classes depend on the critter interpreter working. To do this we manually created multiple Critter Files, including different edge cases, to run our parser through to generate Critter ASTs. Considering we know the expected behavior of the written Critter Files, we observed the outcome of running our AST Interpreter on that Critter ASTs. We used these Critter Files and more Critter Files with randomly generated AST's to also test if we are able to correctly load them with assert statements. Additionally, we utilized our RandomASTGenerator from a4 that generated arbitrarily random ASTs with unpredictable behavior as edge cases for `advanceTime()` method for simulator.

Following this method, we manually created specific critter programs of edge cases for the `step()` method in critter. For example, for mate, we tested if the two different tiles where children can be born (either behind the father or the mother) were covered by food, rock, or other critters, then no children would be born. We would mostly try to use `printWorld()` statements to check the states before and after when actually debugging, to make sure that the correct actions happened. For example, when debugging bud, we realized that the direction was erroneous, and realized that the error stemmed from the communication between our World and Tiles class. That was one of the biggest problems with testing, as for any error, you had to track down the bug through functions and classes that were tightly coupled with each other, which heavily slowed down our debugging process.

In order to thoroughly test all critter actions, we had to manually write many different scenarios and edge cases for every single action.

We tested stepping critters by running the Critter Interpreter on the Critter Files and testing its accuracy and also checking that the world file is updated correctly based on the critter's action taken. We will repeat this for multiple critters of the same world as well.

We will test World and more specifically the Tiles object before & after Critter Action(s) have taken place (for relatively hard actions). We will use assert statements to test the expected state of the Tiles object after a Critter whose AST dictates to only move forward, or an AST that only turns left, etc. We will create multiple Full World files using the Critter Files that we made and test loading in the files using assert statements. We will check that our world is working correctly by observing that the printed ASCII-art world matches the world file and any updates/actions taken by the critters. We will check that Spiral Critter travels in a spiral path by observing its motion on the printed ASCII-art world.

For each critter action, we wrote specialized Test Cases that checked the world state and critter memory was updated as expected. More specifically, we wrote specific test cases for mate and bud since Mating requires stepping two Critters instead of one complicates the implementation and allows for all kinds of issues to arise. For example, we use a small world, like 6*10, and manually create a test case, like mating, failed mating, budding, and insert multiple critters that are in different orientations.

Additionally, we wrote tests for all the Critter sensor functions, allowing us to double check the data Critter is receiving is correct.

Work Plan

For Work Plan, we delegated the following tasks to these following team members:

(Gabriel & Wallace): Implement World & Tiles class.

(Leo & Katherine): Implement Critter class and ASTInterpreter class.

(Leo & Katherine): Write a test suite for World class and its dependencies (except Critter)

(Gabriel & Wallace): Write a test suite for Critter class and its dependencies

(Gabriel): Solve the written problems.

Known Problems

none

Comments

We spent some time wondering if the hex design was worth the struggle of implementing it instead of a square world. We spent most of our time testing & debugging.